Latest updates: https://dl.acm.org/doi/10.1145/3652212.3652216

RESEARCH-ARTICLE

Spatial Publish/Subscribe: Decoupling Game State Dissemination from State Computation for Massive Multiplayer Online Games

PJ SMIT, Stellenbosch University, Stellenbosch, Western Cape, South Africa

HA ENGELBRECHT, Stellenbosch University, Stellenbosch, Western Cape, South Africa

Open Access Support provided by:

Stellenbosch University

PDF Download
3652212.3652216.pdf
04 March 2026
Total Citations: 0
Total Downloads: 408

Published: 15 April 2024

Citation in BibTeX format

MMSys '24: ACM Multimedia Systems Conference 2024
April 15 - 18, 2024
Bari, Italy

Conference Sponsors:
SIGMM

MMVE '24: Proceedings of the 16th International Workshop on Immersive Mixed and Virtual Environment Systems (April 2024)
https://doi.org/10.1145/3652212.3652216
ISBN: 9798400706189

Spatial Publish/Subscribe - Decoupling Game State Dissemination from State Computation for Massive Multiplayer Online Games

PJ Smit
Department of E&E Engineering
Stellenbosch University
South Africa
22573216@sun.ac.za

HA Engelbrecht
Deptartment of E&E Engineering
Stellenbosch University
South Africa
hebrecht@sun.ac.za

ABSTRACT

The rapid expansion of virtual environments, particularly Massively Multi-user Virtual Environments (MMVEs), presents significant challenges in scalability and performance. Traditional Client/Server and Client/Multi-Server architectures often encounter limitations such as server overload, which can lead to lag and reduced user capacity, negatively affecting the user experience. Interest management is an important mechanism in online games for improving scalability, yet it typically involves the server sending duplicate game state update messages for each client impacted by a game state update. This paper introduces an approach that decouples state update dissemination from state computation, enabling the server to focus on state computation while a dedicated server manages the dissemination of state updates to affected clients. Using the VAST architecture, a Spatial Publish/Subscribe (SPS) library, allows for a single update message per game state update to be sent by the server, thereby replacing the traditional interest management scheme. The effectiveness of this approach is experimentally verified through the implementation of SPS on an open-source, high-performance Minecraft server, SpigotMC. Initial implementation and evaluation demonstrate that the VAST architecture effectively reduces computational load and memory usage, while optimising network traffic and latency. For up to six clients, the Minecraft server utilizing SPS exhibits a six-fold decrease in the number of transmitted update messages, marking a substantial reduction in packet transmission compared to traditional methods. This research underscores the potential of Spatial Publish/Subscribe systems in creating more scalable and efficient virtual environments, addressing the evolving demands of virtual world interactions.

CCS CONCEPTS

• **Information systems** → **Data management systems**; **Publish-subscribe / event-based architectures**; • **Human-centered computing** → *Visualization*; • **Theory of computation** → *Online algorithms*.

This work is licensed under a Creative Commons Attribution International 4.0 License.
MMVE '24, April 15–18, 2024, Bari, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0618-9/24/04.
https://doi.org/10.1145/3652212.3652216

KEYWORDS

Spatial Publish/Subscribe, Virtual Environments, Scalability, Multi-server Architecture, Distributed Systems, Minecraft, Network Traffic Optimisation, Latency Reduction

ACM Reference Format:
PJ Smit and HA Engelbrecht. 2024. Spatial Publish/Subscribe - Decoupling Game State Dissemination from State Computation for Massive Multiplayer Online Games. In *16th International Workshop on Immersive Mixed and Virtual Environment Systems (MMVE '24), April 15–18, 2024, Bari, Italy*. ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3652212.3652216

1 INTRODUCTION

The rapid increase of virtual environments, notably in Massively Multi-user Virtual Environments (MMVEs), large-scale simulations, and Massive Multiplayer Online Games (MMOGs), underscores the need for enhanced scalability and performance [9, 35]. These environments, facilitating real-time interaction among widespread users, face challenges in network performance and scalability, particularly when supporting thousands of simultaneous users in expansive worlds.

Traditionally, MMOGs employ centralised Client/Server (C/S) or Client/Multi-Server (C/MS) architectures, handling game state computations and user interactions centrally [35]. While relatively simple, these architectures struggle with scalability, leading to server overload, communication latency, and restricted user capacity, negatively impacting user experience. To mitigate this, Interest Management (IM) [3, 24] systems limit the game state accessible to a user, confined to their Area of Interest (AOI) [17, 30], but this still binds the server capacity to the number of users within an AOI.

Addressing these constraints, we propose a distributed architecture, separating game state update dissemination from computation. We integrate Spatial Publish/Subscribe (SPS) [14], an extension of the Publish/Subscribe model with a spatial dimension, to replace traditional IM. This approach of employing a single state update message for each state change, which is disseminated by an SPS broker, significantly improves scalability.

This paper explores the feasibility of substituting a commercial MMOG's IM with an SPS-based system, specifically in Minecraft, by decoupling the game state update dissemination from the game state computation. Using VAST, an open-source SPS communication library [26], we replaced Minecraft's AOI system with a SPS-based IM system. The concept is experimentally verified using Koekepan [10, 11], a research platform that extends SpigotMC (an open-source Minecraft server clone) [32] to support server clusters.

Our contributions are twofold: introducing the SPS architecture to improve MMOG scalability and performance, demonstrated using

Minecraft, and providing experimental evidence of its efficiency in state update dissemination with comparable latency to existing C/S models.

2 BACKGROUND

2.1 Massively Multi-user Virtual Environments

MMVEs, particularly MMOGs, support interaction among thousands to tens of thousands of users through avatars in a shared virtual world [35]. MMOGs can be seen as state machines, where game state changes (such as entity property changes), are updated through state update messages sent to clients. Changes in game state are caused by internal logic or user interactions, and are managed through a cycle of *event*, *processing*, and *update*. This cycle involves state update messages being disseminated to clients after server processing ensures game logic compliance, known as state computation. Packets, the primary data transmission units in networks, are crucial for communicating state changes. Efficient packet management is key for real-time interaction in VEs, as they carry essential metadata for routing and processing within the network.

The MMOG networking architecture must scale to handle dynamic user volumes. The prevalent Client/Server (C/S) architecture features a central server maintaining the Global State of the game world, with clients' Local State being updated through state dissemination. However, this architecture faces scalability limitations due to the high cost and limitations on upgrading server processing and network communication capacity.

Client/MultiServer (C/MS) architectures distributes the Global State across multiple servers, whereas the Peer-to-Peer (P2P) model utilizes each node as both client and server, sharing server computations. Hybrid architectures combine elements of these systems, like using a central server for critical functions and P2P for less crucial data, achieving a balance of control and scalability. Examples such as the Distributed Scene Graph [18] demonstrate the effectiveness of decoupling system components in hybrid models.

2.2 Interest Management

Interest Management (IM) is essential for scalability in multiplayer games, acting as a data filtration system to manage state dissemination by limiting server-to-client updates, thus reducing network traffic and packet transmission [13, 20, 24]. This restriction on data access is based on the concept that players have limited sensing, i.e. limited movement and vision based on their Area-Of-Interest (AOI), a spatial area surrounding them. This means that game data has both spatial and temporal locality [19], and leads to restricted interaction capabilities outside a players immediate vicinity.

A common IM strategy is *zoning* or *spatial partitioning*, dividing the game world into zones [6]. The player's AOI can be the entire zone the player is located in (i.e. the zone is the entire VE available to the player) or a subarea within it, often visualised as a circle or sphere centred on the player avatar's location. The AOI adjusts when the player moves, with the server updating the game state relevant to a player's AOI.

In C/MS architectures, a common issue is that players on the borders of zones must still be able to see and interact with objects that are just across the zone boundary in a neighbouring zone. This requires more advanced interest management schemes that allow the AOI to overlap multiple zones. Managing AOIs across zone borders presents challenges like object replication [35], necessitating solutions like *mirroring*, where servers replicate game object states across neighboring zones [8, 21].

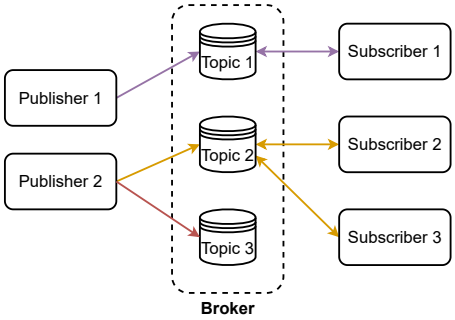


Figure 1: A Publish/Subscribe (Pub/Sub) network, where publishers and subscribers interact through topics in a decoupled system.

Despite its widespread adoption in MMOGs since its inception [13], IM has seen little innovation until 2009, when Hu [14] made a proposal to replace AOI-based IM with Spatial Publish/Subscribe (SPS). SPS, an extension to the Publish/Subscribe messaging paradigm, allows for the decoupling of game state update dissemination from game state computation, effectively creating two distinct layers for IM and the VE hosting. SPS facilitates dynamic resource allocation and removes IM overhead from VE hosting. This paper validates the SPS-based IM scheme first proposed by [14].

3 RELATED WORK

Yahyavi and Kemme [35] presented a comprehensive review of interest management (IM) in MMOGs, highlighting the need for efficient state synchronization in distributed virtual environments. Their work underscored the significance of spatially-aware Pub/Sub systems for enhancing scalability and reducing network traffic. More recently [29] presented an survey of AOI management in MMOGs, highlighting that IM is a core activity. Bharambe presents Donnybrook [4], a P2P system that enables MMOGs with hundreds of simultaneous players. Donnybrook reduces server bandwidth demands by using "interest sets" to model players' limited attention and disseminate frequent updates to only a small set of peers each player is focused on. It handles heterogeneity in peer capacity by using a dynamic forwarding pool where high-capacity peers assist lower-capacity peers.

The idea of using Publish/Subscribe messaging for IM was first suggested by Morgan [23]. However, its topic-based approach falls short for spatially-driven applications like MMOGs. Addressing this, Hu [14] proposed Spatial Publish/Subscribe (SPS) as a foundational element for Virtual Environment (VE) systems. SPS enables nodes in VEs to subscribe and broadcast within specific areas, refining message dissemination based on spatial relevance.

Buyukkaya and Abdallah [5] introduced Voronoi-based spatial partitioning in a fully distributed P2P architecture while also dealing with data management for mutable and immutable objects. In 2016 Abdulazeez proposed a static AOI management scheme and

evaluated using simulation in OPNET Modeler 18.0 to simulate up to 1000 nodes [2] and in 2017 the static interest management scheme was made dynamic [1].

Perhaps the most similar recent work is Cloud Imperium Games' Star Citizen. It aims to enable massive-scale multiplayer spaceship combat through what they call Server Meshing. This interconnects game servers to distribute load by making use of a Replication Layer which players are connected to and allows seamless transition between server instances and entity syncing. All data gets passed through the replication layer to appropriate servers and clients. Recent improvements have introduced Persistent Entity Streaming (PES) across servers (via the replication layer) and allow servers to unload data of neighbouring servers that are not applicable or within view of the server's geographic area. The replication layer makes use of Object Container Streaming, which only transmits the necessary subset of game data between servers. Although promising, server meshing currently only supports basic static meshing and the technology is still in the early stages[27, 28].

To our knowledge, none of the related work addresses the issue that update dissemination using AOI management still scales linearly with the number of affected clients.

4 SPATIAL PUBLISH/SUBSCRIBE ARCHITECTURE (SPS)

The Publish/Subscribe (Pub/Sub) architecture, crucial to SPS, provides a flexible alternative to traditional network architectures like the C/S model for distributing information [12, 31]. In the basic Pub/Sub system, illustrated in Fig. 1, publishers send messages to a broker with a specific topic, and subscribers receive messages of their subscribed topics, maintaining anonymity between them. This model offers network scalability and supports event-driven architectures through time and space decoupling between publishers and subscribers.

However, the traditional Pub/Sub architecture, despite being beneficial in IoT applications such as MQTT [25], falls short in MMOGs due to its topic-based messaging. It does not accommodate the spatial and temporal locality inherent in MMOG packets. Some Pub/Sub protocols do offer geo-support but are restricted to specific locations and don't acommodate generic spatial locations [26]. Additionally, Pub/Sub architectures typically don't natively support multiple brokers, and often lack efficient message forwarding algorithms or a partitioning overlay network for load management.

To overcome these spatial limitations, Hu introduced SPS, which integrates spatial information into the Pub/Sub paradigm [14, 16], making it suitable for spatially dynamic applications such as MMOGs. As shown in Fig. 2, participants (publishers, subscribers and brokers) define a Area Subscription or Point Subscription, with spatial messages routed to clients whose Subscriptions intersect with a Area Publication or Point Publication. SPS supports four operations: (1) PUBLISH messages to a spatial area, (2) SUBSCRIBE to messages in a spatial area, (3) UNSUBSCRIBE from a spatial area, and (4) MOVE the participant's location. Hu [14] also provides a brief overview of how SPS can be used in the context of MMOGs.

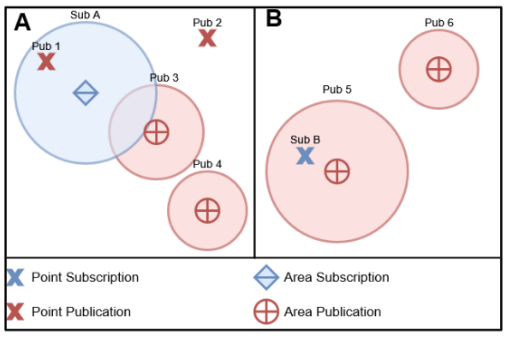


Figure 2: The operational framework of Spatial Publish/Subscribe (SPS) within a virtual environment.

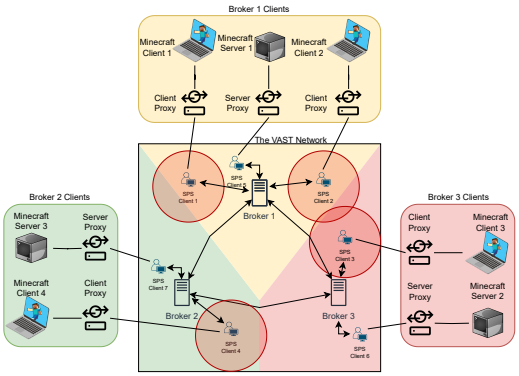


Figure 3: The VAST network implementation in Minecraft.

4.1 VAST

The VAST network library, developed in Javascript, is an implementation of the SPS paradigm, designed to enhance communication in virtual environments [26]. VAST introduces multiple brokers, termed Interest Matchers, each overseeing a specific region in the Virtual Environment (VE). These regions are organised through Voronoi Partitioning within the Voronoi Overlay Network (VON), optimising load distribution and preventing node overload [15, 33]. The VoroCast algorithm is employed for efficient message forwarding to the appropriate broker when publications fall outside a broker's region, creating a spanning tree for seamless inter-broker communication [7].

VAST operates on a peer-to-peer (P2P) framework, allowing nodes to dynamically join, leave, or change their positions within the network. Its communication is based on spatial relevance, ensuring that only relevant brokers and clients process and communicate pertinent events. The library supports the fundamental SPS operations: publications (transmission of spatially relevant information) and subscriptions (receiving updates on publications within a specific spatial area), with brokers accomodating these roles based on client locations and interests.

Fig. 3 demonstrates VAST's application in a C/MS MMOG architecture, using Minecraft as an example. Minecraft clients and servers correspond to SPS clients on the network. Each SPS client

interacts only with the broker in their area, with Minecraft client subscriptions (their AOI) depicted as red circles around SPS Clients. Minecraft servers subscribe to the entire area under their broker's responsibility. Spatial messages published by SPS clients are forwarded by the relevant broker to the appropriate client, possibly via other brokers. Further details about VAST and its use of multiple SPS brokers are available in [26].

5 METHODOLOGY

In this section we introduce the popular commercial MMOG Minecraft, which is used to experimentally evaluate the proposed SPS-based IM scheme. After discussing the technical details of Minecraft, we delve into the practical implementation of the VAST SPS library within a Minecraft multiplayer server environment. We briefly discuss the system architecture and the modifications made to the standard Minecraft C/S model for VAST integration.

5.1 Minecraft

Minecraft is a sandbox-style MMVE game developed by Mojang Studios [22], which operates within a procedurally generated 3D environment comprised of block elements. Its VE is divided into "chunks" (16x16 block areas, extending vertically up to 265 blocks), forming the basic unit of the game's spatial structure.

In multiplayer mode, Minecraft adopts a C/S model [34]. The server holds the *global game state*, managing all aspects of the game world, including chunks, block types, entities, and other elements. Clients connect to this server, receiving data and state changes necessary to render their *local game state* – essentially, the immediate surroundings they interact with.

Minecraft's IM system is designed to optimize network traffic and server load by transmitting information based on players' locations and actions, essentially an AOI system. For example, only chunks within a player's view are sent to their client, minimizing network data transmission.

Minecraft's architecture mirrors that of typical MMOGs, featuring expansive, interactive worlds. It, however, uniquely allows extensive player-driven modification of both game environment and mechanics. This flexibility makes Minecraft a suitable platform for networking research in MMVEs. However, its non-open-source nature poses challenges for modifying core game components.

5.1.1 Bukkit and Koekepan. To enhance Minecraft's functionality, the community has developed tools like Bukkit, an open-source API framework, and CraftBukkit, an open-source server clone. Bukkit offers event-based software hooks into the Native Minecraft Server (NMS) code, enabling third-party 'mods' to interact with events processed by the server. SpigotMC, a high-performance version of CraftBukkit [32], optimizes NMS server code by improving entity handling, chunk loading, and packet transmission.

In previous work we proposed Koekepan [10, 11], an extension to SpigotMC, enhancing the Minecraft server's networking for distributed server architectures. It uses spatial partitioning with Voronoi diagrams to allocate server regions and introduces a proxy between Minecraft clients and servers, facilitating entity migration and load distribution.

5.2 System Architecture and Design

Integrating the VAST library into Minecraft's multiplayer environment necessitated adapting the existing C/S model to accommodate SPS mechanisms while preserving game functionality.

Integration into Minecraft: The integration involved two key steps: removing Minecraft's existing IM and implementing an SPS Broker for packet management. Minecraft uses TCP connections for C/S communication. We broadly catagorise the packets as Spatial, Player Specific, and Global based on their spatial relevance [34].

We modified the Koekepan architecture, dividing the single proxy into separate server and client proxies, connected via an SPS broker, as shown in Fig. 3. The server proxy manages TCP connections to the server for each client, forwarding packets without direct client-server connections. The client proxy similarly manages server message transmission to clients. This setup allows for the indirect exchange of packets via the SPS broker.

In the original architecture, the server broadcasts game state updates to all relevant clients, identified by their AOI. With SPS-based IM, the server publishes a single update, offloading IM from the NMS code to the SPS Broker, thus requiring modifications to the SpigotMC and Koekepan server applications. Additionally, we introduced a dedicated TCP connection between the Minecraft server and server proxy for non-client-specific SPS Publications, such as spatial packets.

The resultant SPS-Koekepan server architecture, depicted in Fig. 3, integrates SPS-based IM within Minecraft's architecture.

System Components: Key components in this architecture (Fig. 3) include the SPS broker (VAST Matcher), handling subscriptions and publications based on spatial dynamics, and corresponding server and client proxies for Minecraft. The server proxy translates between Minecraft and SPS packets, managing spatial packet publication. It also maintains a TCP connection with the server for transmitting spatial packets.

Client proxies serve dual roles: interfacing between the Minecraft client and the SPS network, and managing client-specific SPS interactions. Each client proxy connects to an individual SPS client, representing the Minecraft client in the SPS network, and interfaces with the Broker for publications and subscriptions.

6 EXPERIMENTAL VALIDATION

6.1 Experimental Setup

Our experimental setup includes a single Minecraft server and up to six clients, all running on a host equipped with a 12-core Intel i7-12700 (4.8GHz) CPU and 32GB DDR4 memory, using POP!_OS based on Debian 22.04.

We use two baseline systems: the Java native Minecraft server (NMS) version 1.11.2 from Mojang Studios and the open-source, high-performance SpigotMC server version 1.11.2, which includes the NMS code. Both baselines implement AOI Management within the NMS code. The version number refers to the Minecraft communication protocol for server-client information exchange.

Our system, termed SPS-Koekepan, is a modified version 1.11.2 SpigotMC server integrating the SPS interest management scheme. We employ a single SPS broker, although VAST supports multiple brokers. The evaluation involves three server configurations: NMS,

SpigotMC, and SPS-Koekepan. The simulation procedure for each configuration is as follows:

(1) Initialize all system components: server, clients, proxies, and broker.
(2) Connect from 1 to a maximum of 6 clients, which then navigates the environment for 120 seconds. All clients stay predominantly within each others' AOI.
(3) Disconnect all clients from the server.
(4) Continue system operation for an additional 120 seconds to collect post-operation performance data.
(5) Terminate all system components.

The "Superflat" Minecraft world is used, which does not influence the size of the chunk data sent to the clients, with non-player entities roaming. The presented results are the average of 10 simulation runs for each experimental setup. A client emulator replicates avatar movements and packet transmissions for consistency. We use a client emulator that implements the version 1.11.2 communication protocol, allowing each client to be programmed to repeat the same avatar movement in each simulation. The client emulator sends the same packets during each simulation.

The evaluation focusses on update dissemination efficiency, measuring total messages sent from the server to clients. Fewer transmissions indicate better network efficiency and reduced duplication of state updates. Additionally, we assess the latency added by SPS, with less than 100 ms being acceptable for MMOGs, and evaluate the SPS broker's CPU and memory usage using Linux's process status (*ps*) application, providing insights into scalability.

6.2 Experimental Results

6.2.1 Update Dissemination Efficiency. Analysis of the total server packet transmissions for the two baseline configurations, NMS and SpigotMC, reveals a proportional increase in transmissions with the number of connected clients. Notably, the NMS configuration consistently sends more transmissions than SpigotMC. For example, with six clients connected, the NMS server sends approximately 300,000 packets, while the SpigotMC server transmits around 100,000, indicating that SpigotMC requires roughly one third the number of packets compared to NMS. This reduction is attributed to SpigotMC's efficient handling of entity movement, minimising redundant packet transmissions.

On the contrary, the SPS-Koekepan configuration demonstrates even greater efficiency. With six clients, it averages around 50,000 packets, six times less than NMS and half of what SpigotMC transmits. This efficiency becomes more pronounced with an increasing number of clients, suggesting the superiority of the SPS scheme in update dissemination. This is starkly illustrated when the server packet transmission is compared in Fig. 4. SPS-Koekepan achieves this by sending a single packet per state update, compared to the multiple duplicate packets necessary for NMS and SpigotMC.

Interestingly, for a single connected client, both SPS-Koekepan and SpigotMC transmit a similar number of packets, aligning with expectations since SPS-Koekepan is an extension of SpigotMC and thus should transmit the same number of server packets when only a single client is connected. However, as client numbers increase to six, SPS-Koekepan maintains its efficiency, transmitting approximately half the packets of SpigotMC and one-sixth of NMS. This

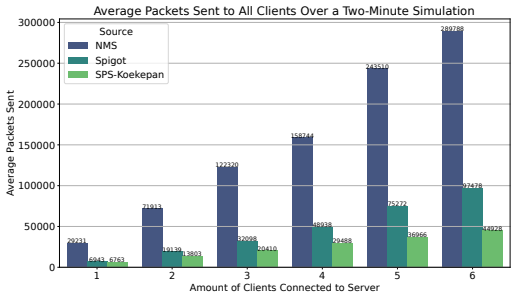


Figure 4: Comparison of Total Server Packet Transmissions for NMS, SpigotMC and SPS-Koekepan configurations

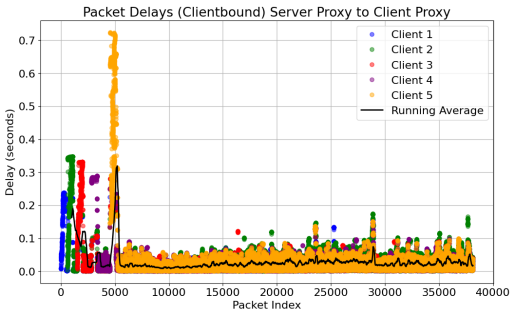


Figure 5: Latency variations with packet delivery for five clients connected to a server.

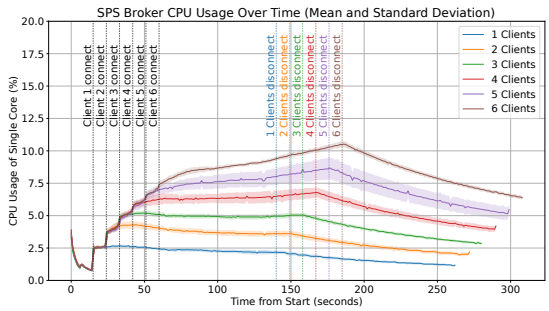
pattern, representing the SPS scheme's update dissemination efficiency, is highlighted in Fig. 4, which summarizes the server packet transmissions across the different server configurations.

The data clearly show that while an increase in connected clients results in more server packet transmissions for all configurations, the rate of increase is substantially lower for SPS-Koekepan. This trend underscores the enhanced efficiency of the SPS-based scheme, particularly as the client count rises.

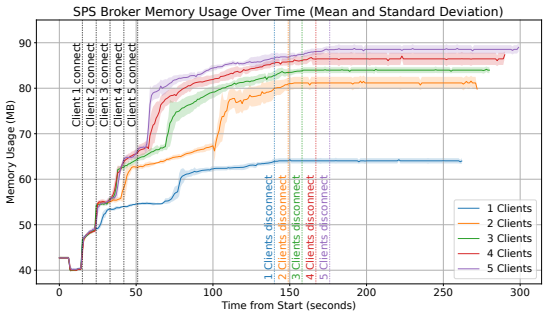
6.2.2 Latency Introduced by SPS scheme. We assess the communication latency added by the SPS scheme to server packets sent to clients. Latency is measured by timestamping network packets, with all simulations on the same host for accurate latency measurements. The latency, from server packet transmission to client reception, is reported for up to five clients connected to a single SPS-Koekepan server.

Fig. 5 shows the average latency for five clients (Client-1 to Client-5) during simulations. Each data point reflects the additional latency from the SPS broker, server and client proxies, and SPS clients, as network packets are transmitted from the SPS-Koekepan server to the clients. The solid black line indicates the running average latency, representing the typical additional latency introduced by the SPS scheme. The packet index is increased each time the server transmits a packet to a client, thus the packet index represents the ordered sequence of sent packets to an individual client.

An initial latency 'spike' is noted for each client at the start of their connection session, particularly within the first 5000 packets. This spike results from the server sending large chunk packets



(a) SPS Broker CPU usage



(b) SPS Broker Memory usage

Figure 6: Average resource utilization of the SPS broker for each client scenario, after 5 simulation runs. Standard deviation is indicated in solid colours.

containing game state data for Minecraft blocks when clients first connect. Each chunk packet consists of the game state data for a column of 16x16x256 Minecraft blocks, and the server sends about 441 chunk packets to each of the clients when they connect. The chunk packets are needed so the client has a local copy of the game state of the Minecraft virtual world. The transmission of these chunk packets explains the increased latency initially observed of up to 700ms. Once the chunk packet transmission is completed, the latency introduced by SPS decreases and stabilizes under 100ms. Occasional peaks in latency are likely due to player movements and requests for more chunk data. A running average latency of around 20ms suggests that, even with five connected clients, the additional latency from SPS is negligible and well below the 100ms MMOG threshold. Considering separate host deployment, an additional 80ms is available for network-induced latency.

6.2.3 Computational and Storage Demand of SPS Broker. The SPS-based scheme's decoupling of game state update computation from dissemination assigns the latter responsibility to the SPS broker. We examined the computational and memory demands of the SPS broker, focusing on CPU and memory usage. Figs. 6a and 6b display the broker's average CPU and memory usage over time, respectively, for one to six clients across five simulations.

Fig. 6a illustrates the CPU usage spike upon each client's connection, indicating an approximate 2% CPU usage increase per client for our test hardware. When clients disconnect, a corresponding decrease in CPU usage is noticeable. Fig. 6b highlights the broker's memory usage, which also rises with client numbers. However, the incremental memory increase per client diminishes with more connections, hinting at a potential plateau in memory demand, however an experiment with significantly more connected clients is needed to verify this conclusion.

The data indicate that update dissemination burden placed on the SPS broker is not significant, with a maximum of 10% CPU usage for one core and no more than 90MB memory for six clients. Assuming linear growth in CPU demand, as Fig. 6a implies, the broker could theoretically support up to 60 clients on a single core.

7 CONCLUSION AND FUTURE WORK

In this paper, we have presented the Spatial Publish/Subscribe (SPS) interest management scheme, which allows us to decouple the game state dissemination from the game state computation performed by a server within the context of Massively Multiplayer Online Games (MMOGs). We highlighted the limitations of traditional area-of-interest management in commercial MMOGs and introduced SPS, integrating spatial information into state updates. This paper, building on Hu's proposal of 2009 [14], provides the first experimental validation of SPS using Minecraft, a widely played commercial MMOG.

Our experiments demonstrated that for six connected clients, SPS significantly reduces server packet transmission by up to six times compared to the native Minecraft server. Moreover, the additional latency introduced by the SPS broker remained below the critical 100ms threshold, averaging around 20ms, thus preserving user experience. The computational and storage demands of the SPS broker were also found to be moderate, with a maximum of 10% CPU usage of a single core and 90MB memory usage, indicating the capacity to handle up to 600 connected clients.

While promising, these preliminary results were obtained with only six clients and should be validated with a larger number of clients running on separate hosts, to also include the effects of network latency. Nevertheless, we are of the opinion that the results do indicate that the SPS-scheme can be used as a replacement for the traditional area-of-interest management scheme.

Future work will focus on validating SPS's performance with a significantly larger number of clients on separate hosts and assessing the impact of a real local area network on latency. Additionally, we plan to test the SPS scheme in the Koekepan architecture, which supports hosting a Minecraft virtual world on a server cluster with up to 120 nodes, using Voronoi-based spatial partitioning. This setup, allowing avatar migration between server nodes without the need for *mirroring* [8, 21], will be a crucial test for SPS's effectiveness in a distributed server environment.

Furthermore, we aim to explore the scalability implications of using multiple SPS brokers, as supported by the VAST implementation. This includes examining the potential increase in transmission latency and, if necessary, considering performance enhancements by reimplementing the VAST library in a more efficient programming

language. These steps will be critical in fully understanding and leveraging the benefits of SPS in large-scale virtual environments.

REFERENCES

[1] Sarmad A. Abdulazeez, Abdennour El Rhalibi, and Dhiya Al-Jumeily. 2017. Dynamic Area of Interest Management for Massively Multiplayer Online Games Using OPNET. In *2017 10th International Conference on Developments in eSystems Engineering (DeSE)*. 50–55. https://doi.org/10.1109/DeSE.2017.19

[2] Sarmad A. Abdulazeez, Abdennour El Rhalibi, and Dhiya Al-Jumeily. 2016. Simulation of Area of Interest Management for Massively Multiplayer Online Games Using OPNET. In *2016 9th International Conference on Developments in eSystems Engineering (DeSE)*. 163–168. https://doi.org/10.1109/DeSE.2016.28

[3] Steve Benford and Lennart Fahlén. 1993. *A Spatial Model of Interaction in Large Virtual Environments*. Springer Netherlands, Dordrecht, 109–124. https://doi.org/10.1007/978-94-011-2094-4_8

[4] Ashwin Bharambe, John R. Douceur, Jacob R. Lorch, Thomas Moscibroda, Jeffrey Pang, Srinivasan Seshan, and Xinyu Zhuang. 2008. Donnybrook: Enabling Large-Scale, High-Speed, Peer-to-Peer Games, In Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication (Seattle, WA, USA). *SIGCOMM Comput. Commun. Rev.* 38, 4, 389–400. https://doi.org/10.1145/1402958.1403002

[5] Eliya Buyukkaya and Maha Abdallah. 2008. Data Management in Voronoi-Based P2P Gaming. In *2008 5th IEEE Consumer Communications and Networking Conference*. 1050–1053. https://doi.org/10.1109/ccnc08.2007.239

[6] Wentong Cai, P. Xavier, S.J. Turner, and Bu-Sung Lee. 2002. A scalable architecture for supporting interactive games on the internet. In *Proceedings 16th Workshop on Parallel and Distributed Simulation*. 54–61. https://doi.org/10.1109/PADS.2002.1004201

[7] Jui-Fa Chen, Wei-Chuan Lin, Tsu-Han Chen, and Shun-Yun Hu. 2007. A forwarding model for Voronoi-based Overlay Network. *2007 International Conference on Parallel and Distributed Systems*, 1–7. https://doi.org/10.1109/ICPADS.2007.4447818

[8] Eric Cronin, Anthony R Kurc, Burton Filstrup, and Sugih Jamin. 2004. An Efficient Synchronization Mechanism for Mirrored Game Architectures. *Multimedia Tools and Applications* 23, 1 (2004), 7–30. https://doi.org/10.1023/B:MTAP.0000026839.31028.9f

[9] Herman A. Engelbrecht and John S. Gilmore. 2017. Pithos: Distributed Storage for Massive Multi-User Virtual Environments. *ACM Transactions on Multimedia Computing, Communications and Applications* 13, 3, Article 31 (6 2017), 33 pages. Issue 3. https://doi.org/10.1145/3105577

[10] Herman A. Engelbrecht and Gregor Schiele. 2013. Koekepan: Minecraft as a research platform, In 2013 12th Annual Workshop on Network and Systems Support for Games (NetGames). *Annual Workshop on Network and Systems Support for Games*, 1–3. https://doi.org/10.1109/NetGames.2013.6820615

[11] Herman A .Engelbrecht and Gregor Schiele. 2014. Transforming Minecraft into a research platform. *2014 IEEE 11th Consumer Communications and Networking Conference (CCNC)*, 257–262. https://doi.org/10.1109/CCNC.2014.6866580

[12] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. 2003. The Many Faces of Publish/Subscribe. *ACM Comput. Surv.* 35, 2 (jun 2003), 114–131. https://doi.org/10.1145/857076.857078

[13] Lennart Fahlen, The Swedish Institute of Computer Science (SICS), and Sweden. 1993. A Spatial Model of Interaction in Large Virtual Environments Steve Benford The University of Nottingham, UK. In *Proceedings of the Third European Conference on Computer-Supported Cooperative Work 13-17 September, 1993, Milan, Italy G De Michelis, C Simone and K. Schmidt (Editors)*.

[14] Shun-Yun Hu. 2009. Spatial Publish Subscribe. *2nd International Workshop on Massively Multiuser Virtual Environments* (9 2009). http://www.badumna.com/

[15] Shun-Yun Hu, Jui-Fa Chen, and Tsu-Han Chen. 2006. VON: a scalable peer-to-peer network for virtual environments. *IEEE Network* 20 (2006), 22–31. https://ieeexplore.ieee.org/document/1668400

[16] Shun-Yun Hu, Chuan Wu, Eliya Buyukkaya, Chien-Hao Chien, Tzu-Hao Lin, Maha Abdallah, Jehn-Ruey Jiang, and Kuan-Ta Chen. 2010. A spatial publish subscribe overlay for massively multiuser virtual environments. *2010 International Conference on Electronics and Information Engineering* 2, V2–314–V2–318. https://doi.org/10.1109/ICEIE.2010.5559789

[17] Björn Knutsson, Honghui Lu, Wei Xu, and Bryan Hopkins. 2004. Peer-to-peer support for massively multiplayer games. In *23rd Annual Joint Conf. of the IEEE Computer and Communications Societies (INFOCOM)*, Vol. 1. 107.

[18] Huaiyu Liu, Mic Bowman, Robert Adams, John Hurliman, and Dan Lake. 2010. Scaling virtual worlds: Simulation requirements and challenges, In Proceedings of the 2010 Winter Simulation Conference. *Proceedings of the 2010 Winter Simulation Conference*, 778–790. https://doi.org/10.1109/WSC.2010.5679112

[19] Yohai Makbily, Craig Gotsman, and Reuven Bar-Yehuda. 1999. Geometric algorithms for message filtering in decentralized virtual environments. 39–46. https://doi.org/10.1145/300523.300527

[20] Tielman Francois Septimus Malherbe. 2016. *A Comparative Study of Interest Management Schemes in Peer-to-Peer Massively Multiuser Networked Virtual Environments*. mathesis. Stellenbosch University, www.eng.sun.ac.za.

[21] Martin Mauve, Stefan Fischer, and Jörg Widmer. 2002. A generic proxy system for networked computer games. In *Proceedings of the 1st workshop on Network and system support for games - NETGAMES '02*. ACM Press, New York, New York, USA, 25–28. https://doi.org/10.1145/566500.566504

[22] Mojang. 2011. What is Minecraft? build, discover realms & more. https://www.minecraft.net/en-us/about-minecraft

[23] Graham Morgan, Fengyun Lu, and Kier Storey. 2005. Interest Management Middleware for Networked Games. In *Proceedings of the 2005 Symposium on Interactive 3D Graphics and Games* (Washington, District of Columbia) *(I3D '05)*. Association for Computing Machinery, New York, NY, USA, 57–64. https://doi.org/10.1145/1053427.1053436

[24] Katherine L. Morse. 1996. Interest Management in Large-Scale Distributed Simulations. In *Technical Report 96-27*.

[25] OASIS. 1999. The standard for IOT messaging. https://mqtt.org/

[26] Victory Opeolu, Herman Engelbrecht, Shun-Yun Hu, and Charl Marais. 2023. VAST: A Decentralized Open-Source Publish/Subscribe Architecture. *Proceedings of the 14th Conference on ACM Multimedia Systems*, 423–429. https://doi.org/10.1145/3587819.3592554

[27] Paul Reindell. 2023. *CitizenCon 2953 Highlight | Server Meshing, PES & Replication Layer*. Youtube | PES. https://www.youtube.com/watch?v=fAbcr35_Teg

[28] Paul Reindell, Benoit Beausejour, Roger Godfrey, and Clive Johnson. 2023. Server Meshing and Persistent Streaming Q&A. https://robertsspaceindustries.com/comm-link/transmission/18397-Server-Meshing-And-Persistent-Streaming-Q-A

[29] Laura Ricci and Emanuele Carlini. 2018. *Area of Interest Management in Massively Multiplayer Online Games*. Springer International Publishing, Cham, 1–3. https://doi.org/10.1007/978-3-319-08234-9_239-1

[30] Arne Schmieg, Michael Stieler, Sebastian Jeckel, Patric Kabus, Bettina Kemme, and Alejandro Buchmann. 2008. pSense - Maintaining a Dynamic Localized Peer-to-Peer Structure for Position Based Multicast in Games. In *2008 Eighth International Conference on Peer-to-Peer Computing*. 247–256. https://doi.org/10.1109/P2P.2008.20

[31] Tarek R Sheltami, Anas A Al-Roubaiey, and Ashraf S Hasan Mahmoud. 2016. A survey on developing publish/subscribe middleware over wireless sensor/actuator networks. *Wireless Networks* 22 (2016), 2049–2070. Issue 6. https://doi.org/10.1007/s11276-015-1075-0

[32] SPIGOTMC. 2012. SPIGOTMC - High Performance Minecraft. https://www.spigotmc.org/

[33] Manrich van Greunen and Herman A. Engelbrecht. 2014. A comparison of Quad-tree and Voronoi-based spatial partitioning for dynamic load balancing. *Proceedings of the first International Conference on the use of Mobile Informations and Communication Technology (ICT) in Africa UMICTA 2014*. http://hdl.handle.net/10019.1/96157

[34] wiki.vg contributors. 2019. Protocol: 1.11.2 Minecraft Protocol Documentation. https://wiki.vg/index.php?title=Protocol&oldid=8543 Online; accessed September 27, 2023, version with oldid=8543.

[35] Amir Yahyavi and Bettina Kemme. 2013. Peer-to-Peer Architectures for Massively Multiplayer Online Games: A Survey. *ACM Computing Surveys (CSUR)* 46 (7 2013). https://doi.org/10.1145/2522968.2522977